

直到运行时才会知道。实际上，在一个 `shared_ptr` 的生存期中，我们可以随时改变其删除器的类型。我们可以使用一种类型的删除器构造一个 `shared_ptr`，随后使用 `reset` 赋予此 `shared_ptr` 另一种类型的删除器。通常，类成员的类型在运行时是不能改变的。因此，不能直接保存删除器。

为了考察删除器是如何正确工作的，让我们假定 `shared_ptr` 将它管理的指针保存在一个成员 `p` 中，且删除器是通过一个名为 `del` 的成员来访问的。则 `shared_ptr` 的析构函数必须包含类似下面这样的语句：

```
// del 的值只有在运行时才知道；通过一个指针来调用它
del ? del(p) : delete p; // del(p) 需要运行时跳转到 del 的地址
```

678 由于删除器是间接保存的，调用 `del(p)` 需要一次运行时的跳转操作，转到 `del` 中保存的地址来执行对应的代码。

在编译时绑定删除器

现在，让我们来考察 `unique_ptr` 可能的工作方式。在这个类中，删除器的类型是类类型的一部分。即，`unique_ptr` 有两个模板参数，一个表示它所管理的指针，另一个表示删除器的类型。由于删除器的类型是 `unique_ptr` 类型的一部分，因此删除器成员的类型在编译时是知道的，从而删除器可以直接保存在 `unique_ptr` 对象中。

`unique_ptr` 的析构函数与 `shared_ptr` 的析构函数类似，也是对其保存的指针调用用户提供的删除器或执行 `delete`：

```
// del 在编译时绑定；直接调用实例化的删除器
del(p); // 无运行时额外开销
```

`del` 的类型或者是默认删除器类型，或者是用户提供的类型。到底是哪种情况没有关系，应该执行的代码在编译时肯定会知道。实际上，如果删除器是类似 `DebugDelete`（参见 16.1.4 节，第 595 页）之类的东西，这个调用甚至可能被编译为内联形式。

通过在编译时绑定删除器，`unique_ptr` 避免了间接调用删除器的运行时开销。通过在运行时绑定删除器，`shared_ptr` 使用户重载删除器更为方便。

16.1.6 节练习

练习 16.28：编写你自己版本的 `shared_ptr` 和 `unique_ptr`。

练习 16.29：修改你的 `Blob` 类，用你自己的 `shared_ptr` 代替标准库中的版本。

练习 16.30：重新运行你的一些程序，验证你的 `shared_ptr` 类和修改后的 `Blob` 类。（注意：实现 `weak_ptr` 类型超出了本书范围，因此你不能将 `BlobPtr` 类与你修改后的 `Blob` 一起使用。）

练习 16.31：如果我们将 `DebugDelete` 与 `unique_ptr` 一起使用，解释编译器将删除器处理为内联形式的可能方式。

16.2 模板实参推断

我们已经看到，对于函数模板，编译器利用调用中的函数实参来确定其模板参数。从函数实参来确定模板实参的过程被称为模板实参推断（`template argument deduction`）。在模

板实参推断过程中,编译器使用函数调用中的实参类型来寻找模板实参,用这些模板实参生成的函数版本与给定的函数调用最为匹配。

16.2.1 类型转换与模板类型参数



679

与非模板函数一样,我们在一次调用中传递给函数模板的实参被用来初始化函数的形参。如果一个函数形参的类型使用了模板类型参数,那么它采用特殊的初始化规则。只有很有限的几种类型转换会自动地应用于这些实参。编译器通常不是对实参进行类型转换,而是生成一个新的模板实例。

与往常一样,顶层 `const` (参见 2.4.3 节,第 57 页)无论是在形参中还是在实参中,都会被忽略。在其他类型转换中,能在调用中应用于函数模板的包括如下两项。

- `const` 转换:可以将一个非 `const` 对象的引用(或指针)传递给一个 `const` 的引用(或指针)形参(参见 4.11.2 节,第 144 页)。
- 数组或函数指针转换:如果函数形参不是引用类型,则可以对数组或函数类型的实参应用正常的指针转换。一个数组实参可以转换为一个指向其首元素的指针。类似的,一个函数实参可以转换为一个该函数类型的指针(参见 4.11.2 节,第 143 页)。

其他类型转换,如算术转换(参见 4.11.1 节,第 142 页)、派生类向基类的转换(参见 15.2.2 节,第 530 页)以及用户定义的转换(参见 7.5.4 节,第 263 页和 14.9 节,第 514 页),都不能应用于函数模板。

作为一个例子,考虑对函数 `fobj` 和 `fref` 的调用。`fobj` 函数拷贝它的参数,而 `fref` 的参数是引用类型:

```
template <typename T> T fobj(T, T); // 实参被拷贝
template <typename T> T fref(const T&, const T&); // 引用
string s1("a value");
const string s2("another value");
fobj(s1, s2);      // 调用 fobj(string, string); const 被忽略
fref(s1, s2);      // 调用 fref(const string&, const string&)
                  // 将 s1 转换为 const 是允许的

int a[10], b[42];
fobj(a, b);        // 调用 f(int*, int*)
fref(a, b);        // 错误: 数组类型不匹配
```

在第一对调用中,我们传递了一个 `string` 和一个 `const string`。虽然这些类型不严格匹配,但两个调用都是合法的。在 `fobj` 调用中,实参被拷贝,因此原对象是否是 `const` 没有关系。在 `fref` 调用中,参数类型是 `const` 的引用。对于一个引用参数来说,转换为 `const` 是允许的,因此这个调用也是合法的。

在下一对调用中,我们传递了数组实参,两个数组大小不同,因此是不同类型。在 `fobj` 调用中,数组大小不同无关紧要。两个数组都被转换为指针。`fobj` 中的模板类型为 `int*`。但是, `fref` 调用是不合法的。如果形参是一个引用,则数组不会转换为指针(参见 6.2.4 节,第 195 页)。 `a` 和 `b` 的类型是不匹配的,因此调用是错误的。



将实参传递给带模板类型的函数形参时,能够自动应用的类型转换只有 `const` 转换及数组或函数到指针的转换。

680

使用相同模板参数类型的函数形参

一个模板类型参数可以用作多个函数形参的类型。由于只允许有限的几种类型转换，因此传递给这些形参的实参必须具有相同的类型。如果推断出的类型不匹配，则调用就是错误的。例如，我们的 `compare` 函数（参见 16.1.1 节，第 578 页）接受两个 `const T&` 参数，其实参必须是相同类型：

```
long lng;
compare(lng, 1024); // 错误：不能实例化 compare(long, int)
```

此调用是错误的，因为传递给 `compare` 的实参类型不同。从第一个函数实参推断出的模板实参为 `long`，从第二个函数实参推断出的模板实参为 `int`。这些类型不匹配，因此模板实参推断失败。

如果希望允许对函数实参进行正常的类型转换，我们可以将函数模板定义为两个类型参数：

```
// 实参类型可以不同，但必须兼容
template <typename A, typename B>
int flexibleCompare(const A& v1, const B& v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

现在用户可以提供不同类型的实参了：

```
long lng;
flexibleCompare(lng, 1024); // 正确：调用 flexibleCompare(long, int)
```

当然，必须定义了能比较这些类型的值的 `<` 运算符。

正常类型转换应用于普通函数实参

函数模板可以有普通类型定义的参数，即，不涉及模板类型参数的类型。这种函数实参不进行特殊处理；它们正常转换为对应形参的类型（参见 6.1 节，第 183 页）。例如，考虑下面的模板：

```
template <typename T> ostream& print(ostream& os, const T &obj)
{
    return os << obj;
}
```

第一个函数参数是一个已知类型 `ostream&`。第二个参数 `obj` 则是模板参数类型。由于 `os` 的类型是固定的，因此当调用 `print` 时，传递给它的实参会进行正常的类型转换：

681

```
print(cout, 42); // 实例化 print(ostream&, int)
ofstream f("output");
print(f, 10);    // 使用 print(ostream&, int); 将 f 转换为 ostream&
```

在第一个调用中，第一个实参的类型严格匹配第一个参数的类型。此调用会实例化接受一个 `ostream&` 和一个 `int` 的 `print` 版本。在第二个调用中，第一个实参是一个 `ofstream`，它可以转换为 `ostream&`（参见 8.2.1 节，第 284 页）。由于此参数的类型不依赖于模板参数，因此编译器会将 `f` 隐式转换为 `ostream&`。



如果函数参数类型不是模板参数，则对实参进行正常的类型转换。

16.2.1 节练习

练习 16.32: 在模板实参推断过程中发生了什么？

练习 16.33: 指出在模板实参推断过程中允许对函数实参进行的两种类型转换。

练习 16.34: 对下面的代码解释每个调用是否合法。如果合法，T 的类型是什么？如果不合法，为什么？

```
template <class T> int compare(const T&, const T&);
(a) compare("hi", "world"); (b) compare("bye", "dad");
```

练习 16.35: 下面调用中哪些是错误的（如果有的话）？如果调用合法，T 的类型是什么？如果调用不合法，问题何在？

```
template <typename T> T calc(T, int);
template <typename T> T fcn(T, T);
double d; float f; char c;
(a) calc(c, 'c'); (b) calc(d, f);
(c) fcn(c, 'c'); (d) fcn(d, f);
```

练习 16.36: 进行下面的调用会发生什么？

```
template <typename T> f1(T, T);
template <typename T1, typename T2> f2(T1, T2);
int i = 0, j = 42, *p1 = &i, *p2 = &j;
const int *cp1 = &i, *cp2 = &j;
(a) f1(p1, p2); (b) f2(p1, p2); (c) f1(cp1, cp2);
(d) f2(cp1, cp2); (e) f1(p1, cp1); (f) f2(p1, cp1);
```

16.2.2 函数模板显式实参



在某些情况下，编译器无法推断出模板实参的类型。其他一些情况下，我们希望允许用户控制模板实例化。当函数返回类型与参数列表中任何类型都不相同时，这两种情况最常出现。

◀ 682

指定显式模板实参

作为一个允许用户指定使用类型的例子，我们将定义一个名为 `sum` 的函数模板，它接受两个不同类型的参数。我们希望允许用户指定结果的类型。这样，用户就可以选择合适的精度。

我们可以定义表示返回类型的第三个模板参数，从而允许用户控制返回类型：

```
// 编译器无法推断 T1，它未出现在函数参数列表中
template <typename T1, typename T2, typename T3>
T1 sum(T2, T3);
```

在本例中，没有任何函数实参的类型可用来推断 T1 的类型。每次调用 `sum` 时调用者都必须为 T1 提供一个**显式模板实参**（**explicit template argument**）。

我们提供显式模板实参的方式与定义类模板实例的方式相同。显式模板实参在尖括号中给出，位于函数名之后，实参列表之前：

```
// T1 是显式指定的, T2 和 T3 是从函数实参类型推断而来的
auto val3 = sum<long long>(i, lng); // long long sum(int, long)
```

此调用显式指定 T1 的类型。而 T2 和 T3 的类型则由编译器从 i 和 lng 的类型推断出来。

显式模板实参按由左至右的顺序与对应的模板参数匹配; 第一个模板实参与第一个模板参数匹配, 第二个实参与第二个参数匹配, 依此类推。只有尾部(最右)参数的显式模板实参才可以忽略, 而且前提是它们可以从函数参数推断出来。如果我们的 sum 函数按照如下形式编写:

```
// 糟糕的设计: 用户必须指定所有三个模板参数
template <typename T1, typename T2, typename T3>
T3 alternative_sum(T2, T1);
```

则我们总是必须为所有三个形参指定实参:

```
// 错误: 不能推断前几个模板参数
auto val3 = alternative_sum<long long>(i, lng);
// 正确: 显式指定了所有三个参数
auto val2 = alternative_sum<long long, int, long>(i, lng);
```

正常类型转换应用于显式指定的实参

对于用普通类型定义的函数参数, 允许进行正常的类型转换(参见 16.2.1 节, 第 602 页), 出于同样的原因, 对于模板类型参数已经显式指定了的函数实参, 也进行正常的类型转换:

683

```
long lng;
compare(lng, 1024);           // 错误: 模板参数不匹配
compare<long>(lng, 1024);     // 正确: 实例化 compare(long, long)
compare<int>(lng, 1024);      // 正确: 实例化 compare(int, int)
```

如我们所见, 第一个调用是错误的, 因为传递给 compare 的实参必须具有相同的类型。如果我们显式指定模板类型参数, 就可以进行正常类型转换了。因此, 调用 compare<long>等价于调用一个接受两个 const long&参数的函数。int 类型的参数被自动转化为 long。在第三个调用中, T 被显式指定为 int, 因此 lng 被转换为 int。

16.2.2 节练习

练习 16.37: 标准库 max 函数有两个参数, 它返回实参中的较大者。此函数有一个模板类型参数。你能在调用 max 时传递给它一个 int 和一个 double 吗? 如果可以, 如何做? 如果不可以, 为什么?

练习 16.38: 当我们调用 make_share(参见 12.1.1 节, 第 401 页)时, 必须提供一个显式模板实参。解释为什么需要显式模板实参以及它是如何使用的。

练习 16.39: 对 16.1.1 节(第 578 页)中的原始版本的 compare 函数, 使用一个显式模板实参, 使得可以向函数传递两个字符串字面常量。



16.2.3 尾置返回类型与类型转换

当我们希望用户确定返回类型时, 用显式模板实参表示模板函数的返回类型是很有效的。但在其他情况下, 要求显式指定模板实参会给用户增添额外负担, 而且不会带来什么好处。例如, 我们可能希望编写一个函数, 接受表示序列的一对迭代器和返回序列中一个

元素的引用：

```
template <typename It>
??? &fcn(It beg, It end)
{
    // 处理序列
    return *beg; // 返回序列中一个元素的引用
}
```

我们并不知道返回结果的准确类型，但知道所需类型是所处理的序列的元素类型：

```
vector<int> vi = {1,2,3,4,5};
Blob<string> ca = { "hi", "bye" };
auto &i = fcn(vi.begin(), vi.end()); // fcn 应该返回 int&
auto &s = fcn(ca.begin(), ca.end()); // fcn 应该返回 string&
```

此例中，我们知道函数应该返回 *beg，而且知道我们可以用 `decltype(*beg)` 来获取此表达式的类型。但是，在编译器遇到函数的参数列表之前，`beg` 都是不存在的。为了定义此函数，我们必须使用尾置返回类型（参见 6.3.3 节，第 206 页）。由于尾置返回出现在参数列表之后，它可以使用函数的参数：

```
// 尾置返回允许我们在参数列表之后声明返回类型
template <typename It>
auto fcn(It beg, It end) -> decltype(*beg)
{
    // 处理序列
    return *beg; // 返回序列中一个元素的引用
}
```

此例中我们通知编译器 `fcn` 的返回类型与解引用 `beg` 参数的结果类型相同。解引用运算符返回一个左值（参见 4.1.1 节，第 121 页），因此通过 `decltype` 推断的类型为 `beg` 表示的元素的类型的引用。因此，如果对一个 `string` 序列调用 `fcn`，返回类型将是 `string&`。如果是 `int` 序列，则返回类型是 `int&`。

进行类型转换的标准库模板类

有时我们无法直接获得所需要的类型。例如，我们可能希望编写一个类似 `fcn` 的函数，但返回一个元素的值（参见 6.3.2 节，第 201 页）而非引用。

在编写这个函数的过程中，我们面临一个问题：对于传递的参数类型，我们几乎一无所知。在此函数中，我们知道唯一可以使用的操作是迭代器操作，而所有迭代器操作都不会生成元素，只能生成元素的引用。

为了获得元素类型，我们可以使用标准库的类型转换（type transformation）模板。这些模板定义在头文件 `type_traits` 中。这个头文件中的类通常用于所谓的模板元程序设计，这一主题已超出本书的范围。但是，类型转换模板在普通编程中也很有用。表 16.1 列出了这些模板，我们将在 16.5 节（第 624 页）中看到它们是如何实现的。

在本例中，我们可以使用 `remove_reference` 来获得元素类型。`remove_reference` 模板有一个模板类型参数和一个名为 `type` 的（public）类型成员。如果我们用一个引用类型实例化 `remove_reference`，则 `type` 将表示被引用的类型。例如，如果我们实例化 `remove_reference<int&>`，则 `type` 成员将是 `int`。类似的，如果我们实例化 `remove_reference<string&>`，则 `type` 成员将是 `string`，依此类推。更一般的，给定一个迭代器 `beg`：

684

C++
11

```
remove_reference<decltype(*beg)>::type
```

将获得 beg 引用的元素的类型：decltype(*beg) 返回元素类型的引用类型。
remove_reference::type 脱去引用，剩下元素类型本身。

组合使用 remove_reference、尾置返回及 decltype，我们就可以在函数中返回元素值的拷贝：

685

```
// 为了使用模板参数的成员，必须用 typename，参见 16.1.3 节（第 593 页）
template <typename It>
auto fcn2(It beg, It end) ->
    typename remove_reference<decltype(*beg)>::type
{
    // 处理序列
    return *beg; // 返回序列中一个元素的拷贝
}
```

注意，type 是一个类的成员，而该类依赖于一个模板参数。因此，我们必须在返回类型的声明中使用 typename 来告知编译器，type 表示一个类型（参见 16.1.3 节，第 593 页）

表 16.1：标准类型转换模板		
对 Mod<T>，其中 Mod 为	若 T 为	则 Mod<T>::type 为
remove_reference	X&或 X&& 否则	X T
add_const	X&、const X 或函数 否则	T const T
add_lvalue_reference	X& X&& 否则	T X& T&
add_rvalue_reference	X&或 X&& 否则	T T&&
remove_pointer	X* 否则	X T
add_pointer	X&或 X&& 否则	X* T*
make_signed	unsigned X 否则	X T
make_unsigned	带符号类型 否则	unsigned X T
remove_extent	X[n] 否则	X T
remove_all_extents	X[n1][n2]... 否则	X T

表 16.1 中描述的每个类型转换模板的工作方式都与 remove_reference 类似。每个模板都有一个名为 type 的 public 成员，表示一个类型。此类型与模板自身的模板类型参数相关，其关系如模板名所示。如果不可能（或者不必要）转换模板参数，则 type 成员就是模板参数类型本身。例如，如果 T 是一个指针类型，则 remove_pointer<T>::type 是 T 指向的类型。如果 T 不是一个指针，则无须进行任何

转换, 从而 `type` 具有与 `T` 相同的类型。

16.2.3 节练习

686

练习 16.40: 下面的函数是否合法? 如果不合法, 为什么? 如果合法, 对可以传递的实参类型有什么限制 (如果有的话)? 返回类型是什么?

```
template <typename It>
auto fcn3(It beg, It end) -> decltype(*beg + 0)
{
    // 处理序列
    return *beg; // 返回序列中一个元素的拷贝
}
```

练习 16.41: 编写一个新的 `sum` 版本, 它的返回类型保证足够大, 足以容纳加法结果。

16.2.4 函数指针和实参推断



当我们用一个函数模板初始化一个函数指针或为一个函数指针赋值 (参见 6.7 节, 第 221 页) 时, 编译器使用指针的类型来推断模板实参。

例如, 假定我们有一个函数指针, 它指向的函数返回 `int`, 接受两个参数, 每个参数都是指向 `const int` 的引用。我们可以使用该指针指向 `compare` 的一个实例:

```
template <typename T> int compare(const T&, const T&);
// pfl 指向实例 int compare(const int&, const int&)
int (*pfl)(const int&, const int&) = compare;
```

`pfl` 中参数的类型决定了 `T` 的模板实参的类型。在本例中, `T` 的模板实参类型为 `int`。指针 `pfl` 指向 `compare` 的 `int` 版本实例。如果不能从函数指针类型确定模板实参, 则产生错误:

```
// func 的重载版本; 每个版本接受一个不同的函数指针类型
void func(int(*) (const string&, const string&));
void func(int(*) (const int&, const int&));
func(compare); // 错误: 使用 compare 的哪个实例?
```

这段代码的问题在于, 通过 `func` 的参数类型无法确定模板实参的唯一类型。对 `func` 的调用既可以实例化接受 `int` 的 `compare` 版本, 也可以实例化接受 `string` 的版本。由于不能确定 `func` 的实参的唯一实例化版本, 此调用将编译失败。

我们可以通过使用显式模板实参来消除 `func` 调用的歧义:

```
// 正确: 显式指出实例化哪个 compare 版本
func(compare<int>); // 传递 compare(const int&, const int&)
```

此表达式调用的 `func` 版本接受一个函数指针, 该指针指向的函数接受两个 `const int&` 参数。

Note

当参数是一个函数模板实例的地址时, 程序上下文必须满足: 对每个模板参数, 能唯一确定其类型或值。

687



16.2.5 模板实参推断和引用

为了理解如何从函数调用进行类型推断，考虑下面的例子：

```
template <typename T> void f(T &p);
```

其中函数参数 `p` 是一个模板类型参数 `T` 的引用，非常重要的一点是记住两点：编译器会应用正常的引用绑定规则；`const` 是底层的，不是顶层的。

从左值引用函数参数推断类型

当一个函数参数是模板类型参数的一个普通（左值）引用时（即，形如 `T&`），绑定规则告诉我们，只能传递给它一个左值（如，一个变量或一个返回引用类型的表达式）。实参可以是 `const` 类型，也可以不是。如果实参是 `const` 的，则 `T` 将被推断为 `const` 类型：

```
template <typename T> void f1(T&); // 实参必须是一个左值
// 对 f1 的调用使用实参所引用的类型作为模板参数类型
f1(i); // i 是一个 int; 模板参数类型 T 是 int
f1(ci); // ci 是一个 const int; 模板参数 T 是 const int
f1(5); // 错误：传递给一个&参数的实参必须是一个左值
```

如果一个函数参数的类型是 `const T&`，正常的绑定规则告诉我们可以传递给它任何类型的实参——一个对象（`const` 或非 `const`）、一个临时对象或是一个字面常量值。当函数参数本身是 `const` 时，`T` 的类型推断的结果不会是一个 `const` 类型。`const` 已经是函数参数类型的一部分；因此，它不会也是模板参数类型的一部分：

```
template <typename T> void f2(const T&); // 可以接受一个右值
// f2 中的参数是 const &; 实参中的 const 是无关的
// 在每个调用中，f2 的函数参数都被推断为 const int&
f2(i); // i 是一个 int; 模板参数 T 是 int
f2(ci); // ci 是一个 const int, 但模板参数 T 是 int
f2(5); // 一个 const &参数可以绑定到一个右值; T 是 int
```

从右值引用函数参数推断类型

当一个函数参数是一个右值引用（参见 13.6.1 节，第 471 页）（即，形如 `T&&`）时，正常绑定规则告诉我们可以传递给它一个右值。当我们这样做时，类型推断过程类似普通左值引用函数参数的推断过程。推断出的 `T` 的类型是该右值实参的类型：

```
template <typename T> void f3(T&&);
f3(42); // 实参是一个 int 类型的右值; 模板参数 T 是 int
```

688

引用折叠和右值引用参数

假定 `i` 是一个 `int` 对象，我们可能认为像 `f3(i)` 这样的调用是不合法的。毕竟，`i` 是一个左值，而通常我们不能将一个右值引用绑定到一个左值上。但是，C++ 语言在正常绑定规则之外定义了两个例外规则，允许这种绑定。这两个例外规则是 `move` 这种标准库设施正确工作的基础。

第一个例外规则影响右值引用参数的推断如何进行。当我们将一个左值（如 `i`）传递给函数的右值引用参数，且此右值引用指向模板类型参数（如 `T&&`）时，编译器推断模板类型参数为实参的左值引用类型。因此，当我们调用 `f3(i)` 时，编译器推断 `T` 的类型为 `int&`，而非 `int`。

`T` 被推断为 `int&` 看起来好像意味着 `f3` 的函数参数应该是一个类型 `int&` 的右值引用。

通常，我们不能（直接）定义一个引用的引用（参见 2.3.1 节，第 46 页）。但是，通过类型别名（参见 2.5.1 节，第 60 页）或通过模板类型参数间接定义是可以的。

在这种情况下，我们可以使用第二个例外绑定规则：如果我们间接创建一个引用的引用，则这些引用形成了“折叠”。在所有情况下（除了一个例外），引用会折叠成一个普通的左值引用类型。在新标准中，折叠规则扩展到右值引用。只在一种特殊情况下引用会折叠成右值引用：右值引用的右值引用。即，对于一个给定类型 X：

- `X& &`、`X& &&`和 `X&& &`都折叠成类型 `X&`
- 类型 `X&& &&`折叠成 `X&&`

C++
11

Note

引用折叠只能应用于间接创建的引用的引用，如类型别名或模板参数。

如果将引用折叠规则和右值引用的特殊类型推断规则组合在一起，则意味着我们可以对一个左值调用 `f3`。当我们将一个左值传递给 `f3` 的（右值引用）函数参数时，编译器推断 `T` 为一个左值引用类型：

```
f3(i);    // 实参是一个左值；模板参数 T 是 int&
f3(ci);   // 实参是一个左值；模板参数 T 是一个 const int&
```

当一个模板参数 `T` 被推断为引用类型时，折叠规则告诉我们函数参数 `T&&` 折叠为一个左值引用类型。例如，`f3(i)` 的实例化结果可能像下面这样：

```
// 无效代码，只是用于演示目的
void f3<int&>(int& &&); // 当 T 是 int& 时，函数参数为 int& &&
```

`f3` 的函数参数是 `T&&` 且 `T` 是 `int&`，因此 `T&&` 是 `int& &&`，会折叠成 `int&`。因此，即使 `f3` 的函数参数形式是一个右值引用（即，`T&&`），此调用也会用一个左值引用类型（即，`int&`）实例化 `f3`：

```
void f3<int&>(int&); // 当 T 是 int& 时，函数参数折叠为 int&
```

这两个规则导致了两个重要结果：

689

- 如果一个函数参数是一个指向模板类型参数的右值引用（如，`T&&`），则它可以被绑定到一个左值；且
- 如果实参是一个左值，则推断出的模板实参类型将是一个左值引用，且函数参数将被实例化为一个（普通）左值引用参数（`T&`）

另外值得注意的是，这两个规则暗示，我们可以将任意类型的实参传递给 `T&&` 类型的函数参数。对于这种类型的参数，（显然）可以传递给它右值，而如我们刚刚看到的，也可以传递给它左值。

Note

如果一个函数参数是指向模板参数类型的右值引用（如，`T&&`），则可以传递给它任意类型的实参。如果将一个左值传递给这样的参数，则函数参数被实例化为一个普通的左值引用（`T&`）。

编写接受右值引用参数的模板函数

模板参数可以推断为一个引用类型，这一特性对模板内的代码可能有令人惊讶的影响：

```
template <typename T> void f3(T&& val)
{
    T t = val; // 拷贝还是绑定一个引用？
```

```

    t = fcn(t); // 赋值只改变 t 还是既改变 t 又改变 val?
    if (val == t) { /* ... */ } // 若 T 是引用类型, 则一直为 true
}

```

当我们对一个右值调用 `fcn` 时, 例如字面常量 `42`, `T` 为 `int`。在此情况下, 局部变量 `t` 的类型为 `int`, 且通过拷贝参数 `val` 的值被初始化。当我们对 `t` 赋值时, 参数 `val` 保持不变。

另一方面, 当我们对一个左值 `i` 调用 `fcn` 时, 则 `T` 为 `int&`。当我们定义并初始化局部变量 `t` 时, 赋予它类型 `int&`。因此, 对 `t` 的初始化将其绑定到 `val`。当我们对 `t` 赋值时, 也同时改变了 `val` 的值。在 `fcn` 的这个实例化版本中, `if` 判断永远得到 `true`。

当代码中涉及的类型可能是普通(非引用)类型, 也可能是引用类型时, 编写正确的代码就变得异常困难(虽然 `remove_reference` 这样的类型转换类可能会有帮助(参见 16.2.3 节, 第 605 页))。

在实际中, 右值引用通常用于两种情况: 模板转发其实参或模板被重载。我们将在 16.2.7 节(第 612 页)中介绍实参转发, 在 16.3 节(第 614 页)中介绍模板重载。

目前应该注意的是, 使用右值引用的函数模板通常使用我们在 13.6.3 节(第 481 页)中看到的方式来进行重载:

```

template <typename T> void f(T&&);           // 绑定到非 const 右值
template <typename T> void f(const T&);      // 左值和 const 右值

```

与非模板函数一样, 第一个版本将绑定到可修改的右值, 而第二个版本将绑定到左值或 `const` 右值。

690

16.2.5 节练习

练习 16.42: 对下面每个调用, 确定 `T` 和 `val` 的类型:

```

template <typename T> void g(T&& val);
int i = 0; const int ci = i;
(a) g(i); (b) g(ci); (c) g(i * ci);

```

练习 16.43: 使用上一题定义的函数, 如果我们调用 `g(i = ci)`, `g` 的模板参数将是什么?

练习 16.44: 使用与第一题中相同的三个调用, 如果 `g` 的函数参数声明为 `T`(而不是 `T&&`), 确定 `T` 的类型。如果 `g` 的函数参数是 `const T&` 呢?

练习 16.45: 给定下面的模板, 如果我们调用一个像 `42` 这样的字面常量调用 `g`, 解释会发生什么? 如果我们调用一个 `int` 类型的变量调用 `g` 呢?

```

template <typename T> void g(T&& val) { vector<T> v; }

```



16.2.6 理解 `std::move`

标准库 `move` 函数(参见 13.6.1 节, 第 472 页)是使用右值引用的模板的一个很好的例子。幸运的是, 我们不必理解 `move` 所使用的模板机制也可以直接使用它。但是, 研究 `move` 是如何工作的可以帮助我们巩固对模板的理解和使用。

在 13.6.2 节(第 473 页)中我们注意到, 虽然不能直接将一个右值引用绑定到一个左值上, 但可以用 `move` 获得一个绑定到左值上的右值引用。由于 `move` 本质上可以接受任

何类型的实参，因此我们不会惊讶于它是一个函数模板。

std::move 是如何定义的

标准库是这样定义 move 的：

```
// 在返回类型和类型转换中也要用到 typename，参见 16.1.3 节（第 593 页）
// remove_reference 是在 16.2.3 节（第 605 页）中介绍的
template <typename T>
typename remove_reference<T>::type&& move(T&& t)
{
    // static_cast 是在 4.11.3 节（第 145 页）中介绍的
    return static_cast<typename remove_reference<T>::type&&>(t);
}
```

这段代码很短，但其中有些微妙之处。首先，move 的函数参数 T&& 是一个指向模板类型参数的右值引用。通过引用折叠，此参数可以与任何类型的实参匹配。特别是，我们既可以传递给 move 一个左值，也可以传递给它一个右值：

```
string s1("hi!"), s2;
s2 = std::move(string("bye!")); // 正确：从一个右值移动数据
s2 = std::move(s1); // 正确：但在赋值之后，s1 的值是不确定的
```

std::move 是如何工作的



在第一个赋值中，传递给 move 的实参是 string 的构造函数的右值结果——string("bye!"). 如我们已经见到过的，当向一个右值引用函数参数传递一个右值时，由实参推断出的类型为被引用的类型（参见 16.2.5 节，第 608 页）。因此，在 std::move(string("bye!")) 中：

- 推断出的 T 的类型为 string。
- 因此，remove_reference 用 string 进行实例化。
- remove_reference<string> 的 type 成员是 string。
- move 的返回类型是 string&&。
- move 的函数参数 t 的类型为 string&&。

因此，这个调用实例化 move<string>，即函数

```
string&& move(string &&t)
```

函数体返回 static_cast<string&&>(t)。t 的类型已经是 string&&，于是类型转换什么都不做。因此，此调用的结果就是它所接受的右值引用。

现在考虑第二个赋值，它调用了 std::move()。在此调用中，传递给 move 的实参是一个左值。这样：

- 推断出的 T 的类型为 string& (string 的引用，而非普通 string)。
- 因此，remove_reference 用 string& 进行实例化。
- remove_reference<string&> 的 type 成员是 string。
- move 的返回类型仍是 string&&。
- move 的函数参数 t 实例化为 string& &&，会折叠为 string&。

因此，这个调用实例化 move<string&>，即

```
string&& move(string &t)
```

这正是我们所寻求的——我们希望将一个右值引用绑定到一个左值。这个实例的函数体返回 `static_cast<string&&>(t)`。在此情况下，`t` 的类型为 `string&`，`cast` 将其转换为 `string&&`。

从一个左值 `static_cast` 到一个右值引用是允许的

**C++
11**

通常情况下，`static_cast` 只能用于其他合法的类型转换（参见 4.11.3 节，第 145 页）。但是，这里又有一条针对右值引用的特许规则：虽然不能隐式地将一个左值转换为右值引用，但我们可以用 `static_cast` 显式地将一个左值转换为一个右值引用。

692

对于操作右值引用的代码来说，将一个右值引用绑定到一个左值的特性允许它们截断左值。有时候，例如在我们的 `StrVec` 类的 `reallocate` 函数（参见 13.6.1 节，第 469 页）中，我们知道截断一个左值是安全的。一方面，通过允许进行这样的转换，C++ 语言认可了这种用法。但另一方面，通过强制使用 `static_cast`，C++ 语言试图阻止我们意外地进行这种转换。

最后，虽然我们可以直接编写这种类型转换代码，但使用标准库 `move` 函数是容易得多的方式。而且，统一使用 `std::move` 使得我们在程序中查找潜在的截断左值的代码变得很容易。

16.2.6 节练习

练习 16.46: 解释下面的循环，它来自 13.5 节（第 469 页）中的 `StrVec::reallocate`：

```
for (size_t i = 0; i != size(); ++i)
    alloc.construct(dest++, std::move(*elem++));
```



16.2.7 转发

某些函数需要将其一个或多个实参连同类型不变地转发给其他函数。在此情况下，我们需要保持被转发实参的所有性质，包括实参类型是否是 `const` 的以及实参是左值还是右值。

作为一个例子，我们将编写一个函数，它接受一个可调用表达式和两个额外实参。我们的函数将调用给定的可调用对象，将两个额外参数逆序传递给它。下面是我们的翻转函数的初步模样：

```
// 接受一个可调用对象和另外两个参数的模板
// 对“翻转”的参数调用给定的可调用对象
// flip1 是一个不完整的实现：顶层 const 和引用丢失了
template <typename F, typename T1, typename T2>
void flip1(F f, T1 t1, T2 t2)
{
    f(t2, t1);
}
```

这个函数一般情况下工作得很好，但当我们希望用它调用一个接受引用参数的函数时就会出现问题：

```
void f(int v1, int &v2) // 注意 v2 是一个引用
{
    cout << v1 << " " << ++v2 << endl;
}
```

在这段代码中，`f` 改变了绑定到 `v2` 的实参的值。但是，如果我们通过 `flip1` 调用 `f`，`f` 所做的改变就不会影响实参：

```
f(42, i);           // f 改变了实参 i
flip1(f, j, 42); // 通过 flip1 调用 f 不会改变 j
```

问题在于 `j` 被传递给 `flip1` 的参数 `t1`。此参数是一个普通的、非引用的类型 `int`，而非 `int&`。因此，这个 `flip1` 调用会实例化为

```
void flip1(void(*fcn)(int, int&), int t1, int t2);
```

693

`j` 的值被拷贝到 `t1` 中。`f` 中的引用参数被绑定到 `t1`，而非 `j`，从而其改变不会影响 `j`。

定义能保持类型信息的函数参数

为了通过翻转函数传递一个引用，我们需要重写函数，使其参数能保持给定实参的“左值性”。更进一步，可以想到我们也希望保持参数的 `const` 属性。

通过将一个函数参数定义为一个指向模板类型参数的右值引用，我们可以保持其对应实参的所有类型信息。而使用引用参数（无论是左值还是右值）使得我们可以保持 `const` 属性，因为在引用类型中的 `const` 是底层的。如果我们将函数参数定义为 `T1&&` 和 `T2&&`，通过引用折叠（参见 16.2.5 节，第 608 页）就可以保持翻转实参的左值/右值属性（参见 16.2.5 节，第 608 页）：

```
template <typename F, typename T1, typename T2>
void flip2(F f, T1 &&t1, T2 &&t2)
{
    f(t2, t1);
}
```

与较早的版本一样，如果我们调用 `flip2(f, j, 42)`，将传递给参数 `t1` 一个左值 `j`。但是，在 `flip2` 中，推断出的 `T1` 的类型为 `int&`，这意味着 `t1` 的类型会折叠为 `int&`。由于是引用类型，`t1` 被绑定到 `j` 上。当 `flip2` 调用 `f` 时，`f` 中的引用参数 `v2` 被绑定到 `t1`，也就是被绑定到 `j`。当 `f` 递增 `v2` 时，它也同时改变了 `j` 的值。



如果一个函数参数是指向模板类型参数的右值引用（如 `T&&`），它对应的实参的 `const` 属性和左值/右值属性将得到保持。

这个版本的 `flip2` 解决了一半问题。它对于接受一个左值引用的函数工作得很好，但不能用于接受右值引用参数的函数。例如：

```
void g(int &&i, int& j)
{
    cout << i << " " << j << endl;
}
```

如果我们试图通过 `flip2` 调用 `g`，则参数 `t2` 将被传递给 `g` 的右值引用参数。即使我们传递一个右值给 `flip2`：

```
flip2(g, i, 42); // 错误：不能从一个左值实例化 int&&
```

传递给 `g` 的将是 `flip2` 中名为 `t2` 的参数。函数参数与其他任何变量一样，都是左值表达式（参见 13.6.1 节，第 471 页）。因此，`flip2` 中对 `g` 的调用将传递给 `g` 的右值引用参数一个左值。



在调用中使用 `std::forward` 保持类型信息

694

我们可以使用一个名为 `forward` 的新标准库设施来传递 `flip2` 的参数, 它能保持原始实参的类型。类似 `move`, `forward` 定义在头文件 `utility` 中。与 `move` 不同, `forward` 必须通过显式模板实参来调用 (参见 16.2.2 节, 第 603 页)。`forward` 返回该显式实参类型的右值引用。即, `forward<T>` 的返回类型是 `T&&`。

C++
11

通常情况下, 我们使用 `forward` 传递那些定义为模板类型参数的右值引用的函数参数。通过其返回类型上的引用折叠, `forward` 可以保持给定实参的左值/右值属性:

```
template <typename Type> intermediary(Type &&arg)
{
    finalFcn(std::forward<Type>(arg));
    // ...
}
```

本例中我们使用 `Type` 作为 `forward` 的显式模板实参类型, 它是从 `arg` 推断出来的。由于 `arg` 是一个模板类型参数的右值引用, `Type` 将表示传递给 `arg` 的实参的所有类型信息。如果实参是一个右值, 则 `Type` 是一个普通 (非引用) 类型, `forward<Type>` 将返回 `Type&&`。如果实参是一个左值, 则通过引用折叠, `Type` 本身是一个左值引用类型。在此情况下, 返回类型是一个指向左值引用类型的右值引用。再次对 `forward<Type>` 的返回类型进行引用折叠, 将返回一个左值引用类型。



当用于一个指向模板参数类型的右值引用函数参数 (`T&&`) 时, `forward` 会保持实参类型的所有细节。

使用 `forward`, 我们可以再次重写翻转函数:

```
template <typename F, typename T1, typename T2>
void flip(F f, T1 &&t1, T2 &&t2)
{
    f(std::forward<T2>(t2), std::forward<T1>(t1));
}
```

如果我们调用 `flip(g, i, 42)`, `i` 将以 `int&` 类型传递给 `g`, `42` 将以 `int&&` 类型传递给 `g`。



与 `std::move` 相同, 对 `std::forward` 不使用 `using` 声明是一个好主意。我们将在 18.2.3 节 (第 706 页) 中解释原因。

16.2.7 节练习

练习 16.47: 编写你自己版本的翻转函数, 通过调用接受左值和右值引用参数的函数来测试它。



16.3 重载与模板

函数模板可以被另一个模板或一个普通非模板函数重载。与往常一样, 名字相同的函数必须具有不同数量或类型的参数。

695

如果涉及函数模板, 则函数匹配规则 (参见 6.4 节, 第 209 页) 会在以下几方面受到